Project Report — Day 1

Amazon Review NLP & Generative AI Web Application

---

1. Project Objective

The objective of this project is to develop an interactive web application that analyzes Amazon e-commerce product reviews using Natural Language Processing (NLP) techniques. The system will gradually evolve from basic data analysis to a conversational AI system powered by Retrieval-Augmented Generation (RAG).

The final application aims to:

- Understand customer sentiment
- Extract useful product insights
- Provide summarized information
- Answer user queries using real reviews

Day-1 focuses on environment setup, dataset exploration, and building an initial Streamlit dashboard.

---

2. Dataset Description

The dataset used is the **Amazon Tech Product Reviews dataset** obtained from Kaggle. It contains real customer reviews for electronic products sold on Amazon.

Main Features

- reviewText — Detailed customer review
- overall — Product rating (1–5 stars)
- summary — Short review title
- asin — Product identifier
- reviewerID — Reviewer identifier
- reviewTime — Date of review

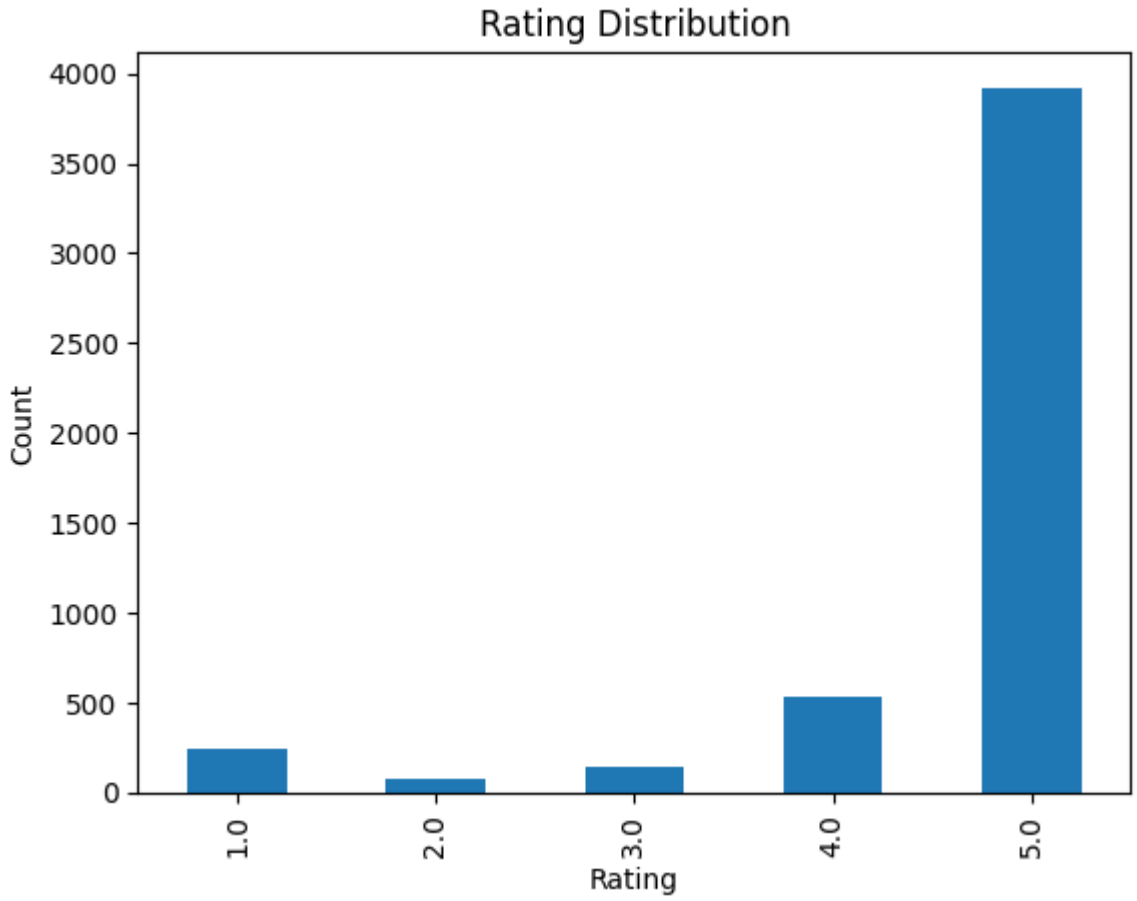
This dataset is suitable for sentiment analysis, text mining, and generative AI applications.

---

3. Exploratory Data Analysis (EDA)

3.1 Rating Distribution

Analysis of rating values shows that most reviews are rated 4 or 5 stars.
This indicates a strong positive bias in customer feedback and introduces class imbalance, which may affect machine learning models in later stages.



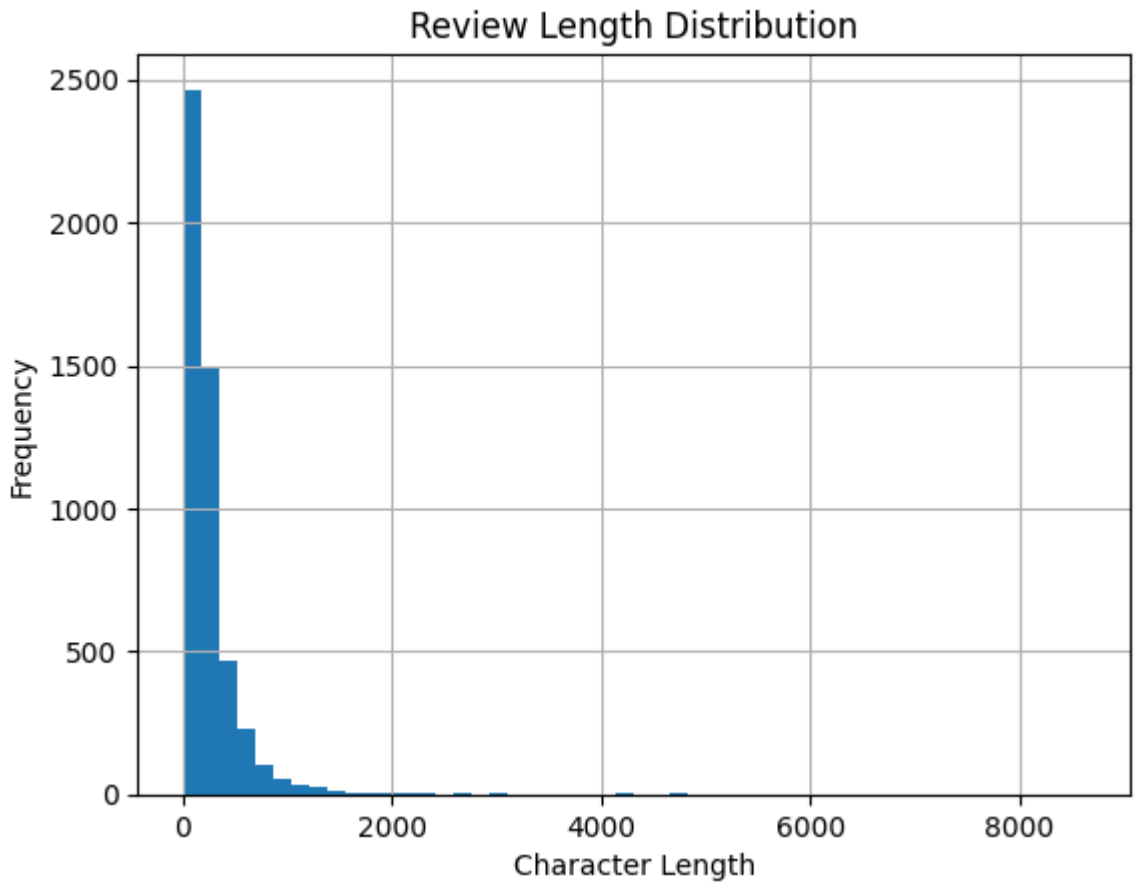
3.2 Review Length Analysis

A new feature review_length was created to measure the number of characters in each review.

Observations:

- Review sizes vary significantly
- Some users provide short feedback such as quick opinions
- Others provide detailed experiences and product evaluations

This variation suggests that future NLP processing must handle both short and long text inputs effectively.



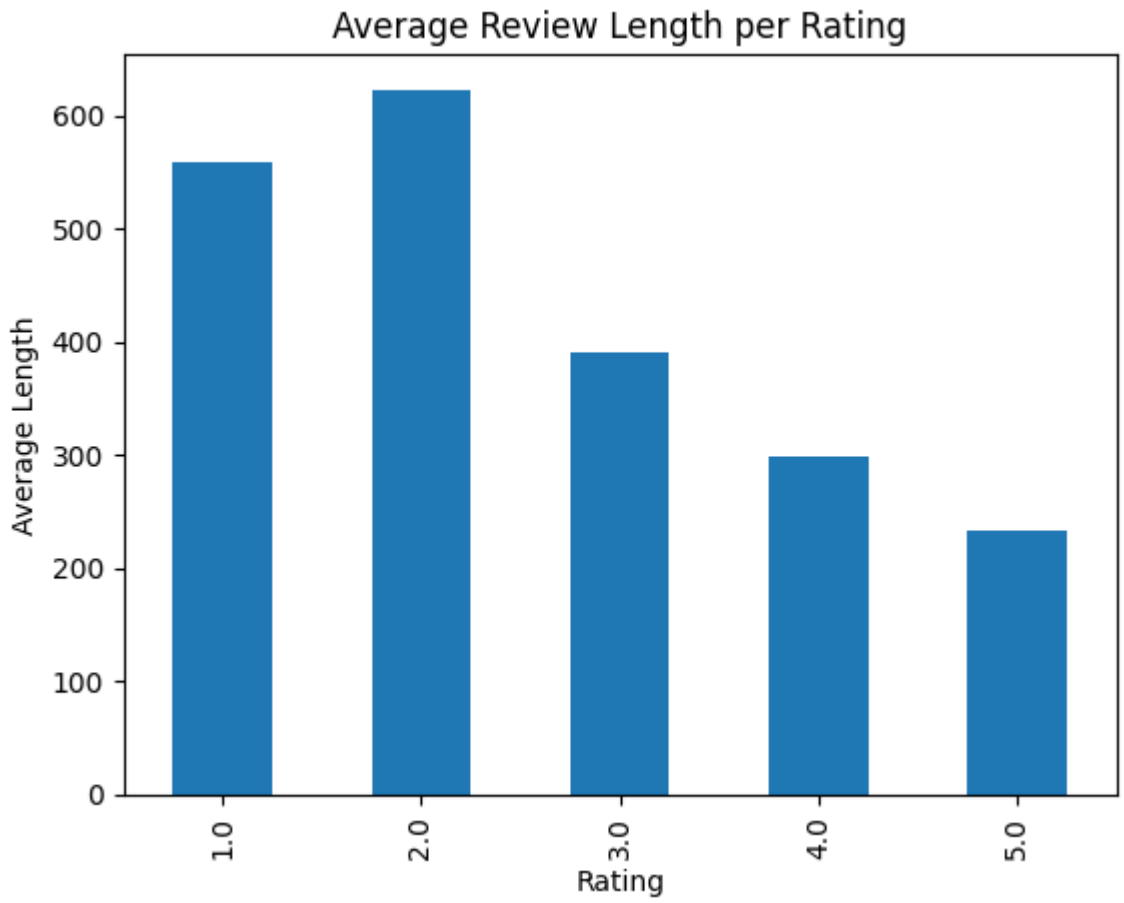
3.3 Rating vs Review Length

The relationship between rating and review length was analyzed.

Key findings:

- Low ratings often have longer reviews because customers explain issues in detail
- High ratings sometimes contain short appreciation comments
- Emotional intensity appears to influence how much users write

This insight will help in feature engineering and also guide chunk size selection when building the future RAG retrieval system.



3.4 Handling Missing Values

Some entries in the reviewText column contained missing values.
They were handled using:

fillna("")

This ensures stable processing and prevents runtime errors during text analysis.

---

4. Streamlit Dashboard

A basic Streamlit application was developed to visualize and interact with the dataset.
The dashboard includes:

- Dataset preview table
- Rating distribution chart
- Review length distribution
- Rating vs review length comparison

This serves as the foundation for adding sentiment prediction and AI-based features in later stages.

---

5. Conclusion

Day-1 successfully established the project environment and performed initial analysis of the dataset. Important behavioral patterns in customer reviews were identified, and a working interactive dashboard was created.

Day 2 – Text Cleaning & Multi-Class Sentiment Model Integration

1. Objective

The goal of Day-2 was to preprocess raw review text and integrate a traditional machine learning model into the Streamlit application for real-time rating prediction. This marks the transition from exploratory analysis to applied NLP modeling.

---

2. Text Cleaning & Preprocessing

Before training the model, the review text was cleaned to reduce noise and improve feature quality. The following preprocessing steps were applied:

- Converted text to lowercase to ensure uniformity
- Removed special characters, punctuation, and numerical noise
- Removed common English stopwords (e.g., *the, is, and*)
- Handled missing values using fillna("") to prevent runtime errors

These steps help reduce irrelevant information and improve the effectiveness of feature extraction. Clean text ensures that the model focuses on meaningful patterns rather than formatting inconsistencies.

---

3. Feature Engineering – Why TF-IDF?

TF-IDF (Term Frequency–Inverse Document Frequency) was used to transform textual data into numerical feature vectors.

TF-IDF was selected because:

- It assigns higher importance to distinctive words
- Reduces the influence of frequently occurring but less informative terms
- Works efficiently with large datasets
- Provides strong baseline performance in text classification tasks
- Is computationally lightweight compared to deep learning models

As a baseline method, TF-IDF is widely used in traditional NLP pipelines before transitioning to embeddings or transformer-based approaches.

---

4. Model Selection – Multinomial Logistic Regression

A multinomial Logistic Regression model was trained to predict review ratings from **1 to 5 stars** instead of binary sentiment.

Logistic Regression was chosen because:

- It performs well with sparse TF-IDF features
- It is fast to train and interpret
- It supports multi-class classification
- It provides probability confidence scores

Class imbalance was addressed using class weighting to improve prediction fairness across rating categories.

---

5. Application Integration

The trained model and TF-IDF vectorizer were saved as serialized .pkl files and loaded into the Streamlit application.

Users can now:

- Enter custom review text
- Receive predicted star ratings (1–5)
- View confidence probabilities for each rating

This completes the first end-to-end NLP pipeline within the application — from raw text input to real-time prediction.

---

6. Conclusion

Day-2 successfully transformed the project from exploratory analysis into a functional NLP application. Text preprocessing was implemented, and a multi-class TF-IDF + Logistic Regression model was trained to predict review ratings. The trained model was integrated into the Streamlit interface, enabling real-time user interaction and prediction.

Day 3 – Zero-Shot Classification using Pre-trained Transformer

1. Concept of Zero-Shot Learning

Zero-Shot learning allows a model to classify text into categories it was never explicitly trained on.
Instead of training a new model with labeled data, we provide possible labels (e.g., *Battery Issue*, *Screen Quality*, *Delivery Speed*), and the pre-trained Transformer determines which label best matches the meaning of the review.
This works because the model has already learned deep language relationships from very large datasets.

---

2. Implementation in the Project

A Hugging Face zero-shot classification pipeline was integrated into the Streamlit app as a new tab.
Users can type or paste any review and instantly receive predicted category tags without training a new dataset-specific model.

---

3. Why It Saves Time Compared to Day-2

On Day-2, we had to clean data, vectorize text (TF-IDF), train a classifier, and save the model.
On Day-3, no dataset labeling or training was required — the pre-trained model directly performs classification.
This significantly reduces development effort while still providing meaningful insights from text.

---

4. Conclusion

The application now supports intelligent tagging of reviews using AI knowledge learned from large-scale language data, demonstrating a faster and more flexible alternative to traditional machine learning approaches.

Day 4 – LLM Integration & Review Summarization (Google Gemini)

1. Objective

The goal of Day-4 was to enhance the application from analysis to reasoning by integrating a Large Language Model (LLM).
Users can now select a product and automatically receive a concise **Pros & Cons summary** generated from real customer reviews.

---

2. Prompt Engineering

Prompt Used

You are an expert product analyst for a MicroSD card.

Analyze the following customer reviews and produce a concise summary.

Return output strictly in this format:

Pros:

- point 1

- point 2

Cons:

- point 1

- point 2

Reviews:

{reviews_text}

This prompt guides the LLM to behave like a product expert and extract only the most important positive and negative aspects from the reviews. The strict format ensures consistent output so it can be directly displayed in the Streamlit interface without additional processing.

---

3. API Integration

The Google Gemini API was connected using the official google-genai SDK.
The application collects the top reviews of a selected product and sends them to the model (gemini-1.5-flash) for summarization.

---

4. Handling Rate Limits & Errors

Since LLM APIs may temporarily fail due to quota or network issues, a retry mechanism was implemented:

- Up to 3 retry attempts
- 2-second delay between attempts
- Graceful fallback message if the model remains unavailable

This prevents the web app from crashing and provides a stable user experience.

---

5. Conclusion

The application now generates intelligent product insights automatically from raw reviews, demonstrating dynamic AI reasoning rather than static machine learning predictions.